WebSocket API

Обновлено 29 мая 2026, 00:43 Андрей Коновалов

General Conventions

- Transport: Socket.IO .
- Auth: access JWT is passed in handshake `auth.token` .
- An invalid token causes an immediate disconnect.
- Room format: `tournament:{tournamentId}` .
- Client messages do not perform domain mutations.
- Domain mutations are performed through REST.
- All server events contain `occurredAt` in ISO 8601 format.
- AsyncAPI is available at `/api-ws` if enabled by configuration.

Connection

Example client flow:

```
const socket = io("http://localhost:3001", {
  auth: {
    token: accessToken,
  },
});
```

After connecting, the client must explicitly join a tournament room:

```
socket.emit("tournament:join", { tournamentId }, (ack) => {
  // { success: true }
});
```

Client-to-Server Events

`tournament:join`

Transport-level subscription to a room.

Payload:

```
{
  "tournamentId": "uuid"
}
```

Ack:

```
{
  "success": true
}
```

Access rules:

- the socket must be authenticated;
- the tournament must exist;
- the user must be a participant;
- the current implementation also checks that there is no unfinished participation elsewhere.

`tournament:leave`

Transport-level unsubscription from a room. This does not remove the participant from the tournament.

Payload:

```
{
  "tournamentId": "uuid"
}
```

Ack:

```
{
  "success": true
}
```

Presence

Presence is stored in Redis:

Key	Purpose
presence:{tournamentId}:sockets:{userId}	user's socket IDs in the tournament
presence:{tournamentId}:users	unique online users in the tournament

Multi-tab semantics:

- one user with multiple sockets is counted as one active user;
- the user is removed from active users only when the last socket in the room is closed.

Disconnect:

- removes the socket from presence;
- sends `tournament:presence_updated`;
- does not remove the participant;
- does not send `tournament:participant_left`.

Server-to-Client Events

`tournament:presence_updated`

```
{
  "tournamentId": "uuid",
  "activeCount": 3,
```

```
  "occurredAt": "2026-05-13T12:00:00.000Z"
}
```

The payload is intentionally lightweight: user IDs are not sent.

`tournament:participant_joined`

Sent after REST join.

```
{
  "tournamentId": "uuid",
  "userId": "uuid",
  "occurredAt": "2026-05-13T12:00:00.000Z"
}
```

`tournament:participant_left`

Sent after REST leave. Socket disconnect does not trigger this event.

```
{
  "tournamentId": "uuid",
  "userId": "uuid",
  "occurredAt": "2026-05-13T12:00:00.000Z"
}
```

`tournament:started`

```
{
  "tournamentId": "uuid",
  "status": "ACTIVE",
  "roundsCount": 3,
```

```
  "occurredAt": "2026-05-13T12:00:00.000Z"
}
```

round:created

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "roundNumber": 1,
  "phase": "SUBMISSION",
  "prompt": {
    "key": "unexpected_superpower",
    "type": "TEXT",
    "content": {
      "en": "Invent an unexpected superpower...",
      "ru": "Придумайте неожиданную суперспособность..."
    }
  },
  "submissionDeadline": "2026-05-13T12:00:30.000Z",
  "occurredAt": "2026-05-13T12:00:00.000Z"
}
```

round:progress_updated

Sent after a submission. Does not contain submission content.

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "phase": "SUBMISSION",
  "submittedCount": 2,
  "totalActiveParticipants": 4,
```

```
  "occurredAt": "2026-05-13T12:00:10.000Z"
}
```

`round:phase_changed`

Used for the `SUBMISSION -> VOTING` transition.

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "roundNumber": 1,
  "previousPhase": "SUBMISSION",
  "currentPhase": "VOTING",
  "occurredAt": "2026-05-13T12:00:30.000Z"
}
```

`voting:submission_revealed`

Reveals one current submission for voting.

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "submission": {
    "id": "uuid",
    "authorId": "uuid",
    "content": "My answer",
    "submittedAt": "2026-05-13T12:00:20.000Z"
  },
  "revealIndex": 0,
  "totalSubmissions": 4,
  "votingDeadline": "2026-05-13T12:01:00.000Z",
  "occurredAt": "2026-05-13T12:00:30.000Z"
}
```

`vote:progress_updated`

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "submissionId": "uuid",
  "votedCount": 2,
  "totalEligibleActiveVoters": 3,
  "occurredAt": "2026-05-13T12:00:40.000Z"
}
```

Progress is calculated from active eligible voters.

`vote:finalized`

Final result for one submission.

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "submissionId": "uuid",
  "likeCount": 2,
  "dislikeCount": 1,
  "occurredAt": "2026-05-13T12:01:00.000Z"
}
```

`round:completed`

```
{
  "tournamentId": "uuid",
  "roundId": "uuid",
  "roundNumber": 1,
  "rankings": [
    {
```

```
      "submissionId": "uuid",
      "authorId": "uuid",
      "likeCount": 2,
      "dislikeCount": 1,
      "score": 2
    }
  ],
  "leaderboard": [
    {
      "userId": "uuid",
      "cumulativeScore": 2,
      "rank": 1
    }
  ],
  "nextRoundNumber": 2,
  "isLastRound": false,
  "occurredAt": "2026-05-13T12:01:05.000Z"
}
```

`tournament:finished`

```
{
  "tournamentId": "uuid",
  "status": "COMPLETED",
  "overallWinnerId": "uuid",
  "finalLeaderboard": [
    {
      "userId": "uuid",
      "cumulativeScore": 6,
      "rank": 1
    }
  ],
  "occurredAt": "2026-05-13T12:03:00.000Z"
}
```

`tournament:cancelled`

```
{
  "tournamentId": "uuid",
  "status": "CANCELLED",
  "occurredAt": "2026-05-13T12:00:00.000Z"
}
```

Error Handling

`WsExceptionFilter` sends an `exception` event only to the requesting socket.

Payload:

```
{
  "message": "Unable to join tournament room"
}
```

Reconnect Flow

```
Socket reconnect with JWT
  -> emit tournament:join
  -> call GET /api/v1/tournaments/:id/full
  -> render REST snapshot
```

The backend does not store or replay WebSocket history.